

Part A — Letting the Terminal ask Niyran AI

Assumptions. The Terminal's platform can hold one secret; if not, this needs a small always-on proxy. Postgres supports row-level security; if not, the wall becomes a public-only replica. The Terminal's own credential is configured for public sources across every constituency, because that is the product; the mechanism below still enforces per-constituency scope, because Dashboard machine credentials come next. Tenancy has two axes today, constituency and party; I assume a credential carries both.

Context

The Terminal reaches an LLM through a proxy anyone can call, with our API key inside the deployed code. We want it to ask Niyran AI instead, so its users get grounded, cited answers — but that knowledge base also holds private field reports covered by DPDP. The change that improves the answers is the one that opens a machine-sized door into that data, in the week the Terminal becomes public. The human Dashboard must keep working untouched.

Decision

Niyran AI gets a second, narrower front door for machines: a new route, `/v1/ask`, authenticated by a bearer token, rate- and budget-limited per token, and served by a database user that physically cannot read what the token is not entitled to. The human path is not modified.

This is also the only place the "light backend" bends. The Terminal's browser is unchanged — no database, no accounts, state in the browser. One serverless function stops being anonymous and holds a secret; the gates' durable state lives on the Dashboard side, which already runs Postgres.

1. The credential

Created by an admin command: 256 random bits, prefixed `niy_`, shown once. **Stored** as an HMAC-SHA256 hash with a pepper — an extra key in the secret manager, never in Postgres — so a leaked database yields hashes of secrets an attacker cannot present. Not bcrypt: password hashes are salted per row, so every call would scan every row, and slowness adds nothing to a 256-bit random secret. **Scoped** by columns on the token's own row: permitted source types, permitted constituency codes, monthly budget. A request may narrow that scope and never widen it — asking outside it is a refusal, not a silent broadening. **Revoked** by stamping the row and checking it every call; rotating the pepper kills every token at once, the panic lever. The Terminal holds one token in its platform's secret store; it never reaches a browser.

2. The money and privacy gates

Both counters live in Postgres, not in process memory — memory is wiped by every deploy and multiplied by the number of server processes. Per token: calls in the last minute, and rupees spent this calendar month, Indian time, because billing months are local. The gate checks before the call and adds the cost after, so a month can end one call over — bounded by the rate limit, and worth the simplicity. Behind our gates sits a hard ceiling on the LLM account itself, set below what we can afford to lose; our own limits can be misconfigured, that one cannot.

Privacy and tenancy are not enforced by an if. The machine route connects to Postgres as its own database user, and a row-level security policy — a rule enforced inside the database, not in our code — restricts that user to chunks tagged public *and* tagged with a constituency the credential is allowed. Both tags, one mechanism: the route publishes the credential's permitted codes to the connection after authenticating. A developer who forgets a filter gets nothing back; neither does a question that talks its way into asking. Retrieval then asserts what came back is in scope and returns nothing if not. Two layers, both failing closed. The Dashboard keeps its existing database user, so nothing there changes.

3. The RAG layer

Index hybrid: a dense vector for meaning plus a keyword index for what embeddings are bad at — section numbers, case citations, party names, Hindi spellings no model has seen. Both halves filter on the two tags *before* scoring, so a Terminal query never ranks an out-of-scope chunk.

Long court judgments split on the document's own structure — headnote, facts, issues, holding, numbered paragraphs — not a word count that cuts sentences in half. Each chunk keeps its paragraph number and page, so a citation points at something a lawyer can open. Over-long paragraphs sub-split with overlap, and matches return with their neighbours.

Short Hindi field reports are usually one chunk each. Store the Hindi as written, embed with a multilingual model so English and Hindi questions reach the same report, and normalise names at ingest, because one village arrives spelled four ways.

Measuring it monthly: freeze about a hundred real questions with a human-checked answer and the ids that support it. Re-run on the first of each month and record three numbers — how often the answer is supported by what was retrieved, how often every cited id exists *and* contains the claim, and how often the system correctly declines. A fall of more than a few points blocks the release, and every bad production answer joins the set — a spreadsheet, not a tool purchase.

4. First week on the Terminal

1. **Treat the deployed keys as already stolen.** Read the provider's usage history *first* — the only way to know whether this is a clean-up or a live incident — then rotate and move the key into the secret store. Everything below is pointless while the old key works.
2. **Close the open AI proxy.** Verify by counting callers in its access log we cannot account for, then require a token. Until it does, strangers spend our money continuously — the one gap with a running meter.
3. **Restrict the URL-fetching proxies.** Verify by asking our own proxy for the cloud metadata address: if it answers, we are one request from handing over the server's credentials. Then allowlist the hosts we read, block private addresses, cap size and time. Third only because the first two are faster — if that fetch succeeds, it goes first.
4. Then the credential, the gates and the database rule above.

Alternatives rejected

A service account on the existing human login. Cheapest, no new code. Rejected: sessions expire on a browser's schedule, and a service account inherits a human's full access — private chunks included.

Filtering private sources in application code — one condition in the retrieval function. Rejected: one forgotten argument from a leak, exactly the mistake in the Part C pull request. "Impossible to get wrong" was the requirement, and a database user that cannot see the rows survives a refactor where an `if` does not.

Counters in Redis, or in the serverless functions. Redis is faster and offloads Postgres. Rejected: a second datastore to run and pay for on a tight budget, and a counter inside a stateless function is not durable at all.

Risks

- **The cap overshoots by at most one call per token per month**, because cost is known only after the call — bounded by the rate limit, and cheaper than reserving up front.
- **The database rule is only as strong as the user it applies to** — Postgres owners bypass it. On startup the service tries to read a known private row and refuses to start if it can.
- **A chunk mis-tagged at ingest defeats every layer**, so ingest needs the scrutiny retrieval gets.